

Submission 1 — Project Proposal

Spring 2026 | Deadline: 15 May 2026

Course	CENG-442 Microservice Architecture
Language	English
Team Size	2 students (mandatory)
Submission	Submission 1 of 2
Deadline	15 May 2026
Instructor	Asst. Prof. Dr. Huseyin Temucin

1. Project Title and Team Members

Project Title: ErasmusMate — An API-First Microservice-Based Platform for International Student Support

Team Members:

- Yago Rodríguez de Pauli - 25118080901
- Ceren Akmeşe – 22118080701

2. Problem Statement

Doing an Erasmus exchange is one of the most chaotic things you can go through as a student, and not in a bad way, but the administrative side of it is genuinely overwhelming. You land in a new country, you have to get your residence permit sorted, figure out which courses actually count for your home university, deal with insurance documents, and on top of all that try to make the most of being abroad. The information you need is never in one place — it's spread across emails, university portals, government websites, WhatsApp groups, random PDFs saved on your laptop, and sometimes just things people told you verbally.

The result is that students miss deadlines, lose track of required documents, or waste time chasing information that should be straightforward to find. A student might not realise their health insurance is about to expire, or forget to book a residence permit appointment because the reminder was buried in an email thread from three weeks ago.

ErasmusMate addresses this directly. The goal is to centralise the most important parts of the Erasmus experience — administrative tracking, document management, academic information, and travel planning — into one modular backend system. The first version is API-first and designed to be tested and demonstrated through Postman. A frontend layer could be added in the future, but the core value of the platform is in the backend logic and service design.

3. System Overview

ErasmusMate is a microservice-based backend platform built specifically for Erasmus and international exchange students. The system provides a set of independent services covering student profile management, residence permit tracking, smart document organisation, academic information, travel planning, and notifications.

What makes this platform different from a basic CRUD application is the Smart Document Management Service, which goes beyond passive storage. It can detect document types, extract relevant metadata such as expiry dates, and proactively alert students when a document is approaching expiration or when a required document is missing. This turns the document layer into an active assistant rather than just a file cabinet.

The system follows an API-first approach. All interactions happen through REST endpoints exposed via an API Gateway, and the platform can be fully tested and demonstrated through Postman. Internally, services communicate through a three-layer model: synchronous REST for client-facing requests, gRPC for high-frequency internal lookups between services, and asynchronous event-driven messaging via a message broker for background processes and notifications (Fowler & Lewis, 2014).

4. Microservice Decomposition

The system is decomposed into six independent microservices, each owning a clearly defined area of responsibility. Services do not share databases and do not call each other directly when it can be avoided. Cross-service interactions happen through REST APIs for client-facing requests, gRPC for high-frequency internal lookups (primarily profile data retrieval), or through the message broker for asynchronous events (Newman, 2021).

1. Student Profile Service

Main responsibility: Manage the student's core identity and Erasmus-related information.

This service is the source of truth for who the student is. It stores basic identity information such as name and nationality, alongside Erasmus-specific data including the home university, host university, exchange period dates, and notification preferences. Other services may query this service through REST when they need context — for example, to personalise deadline calculations or filter relevant announcements. The service does not manage documents, permits, or travel plans; it only owns profile data.

2. Residence Permit Service

Main responsibility: Help students track residence permit procedures and all related deadlines.

One of the most stressful parts of any exchange period is navigating the residence permit or legal stay process. This service allows students to register the current status of their application, log important dates and appointments, and track progress through the different procedural steps. When a deadline is approaching or a status change occurs, the service publishes an event to the message broker so that the Notification Service can alert the student without this service needing to know anything about how notifications are delivered. Possible events include `ResidenceApplicationCreated`, `ResidenceDeadlineApproaching`, `ResidenceAppointmentScheduled`, and `ResidenceStatusChanged`.

3. Smart Document Management Service

Main responsibility: Store, classify, and monitor important documents needed during the Erasmus period.

This service is the most distinctive component of the platform. Rather than functioning as a passive file store, it behaves more like a document assistant. When a student uploads a

document — such as a health insurance certificate, acceptance letter, or learning agreement — the service can detect the document type, extract relevant metadata including expiration dates, and generate warnings when a document is approaching expiry or a required document has not yet been uploaded. This transforms what could have been a simple CRUD service into something genuinely useful. Supported document types include passports, acceptance letters, learning agreements, health insurance documents, student certificates, visa documents, residence permit forms, and enrollment confirmations. Events published by this service include DocumentUploaded, DocumentTypeDetected, DocumentExpiryDetected, DocumentExpiryApproaching, RequiredDocumentMissing, and DocumentVerified.

4. University Services Service

Main responsibility: Centralise academic and university-related information relevant to international students.

International students often struggle to stay on top of academic updates at a university that is not their own. This service works as an academic information hub, managing course information, exam dates, classroom changes, professor office hours, and general university announcements. The goal is to give students a single, reliable place to check what is happening academically without having to monitor multiple platforms or email accounts. Events published include UniversityAnnouncementCreated, ExamDateChanged, ClassroomChanged, and OfficeHourUpdated.

5. Travel Planning Service

Main responsibility: Support travel planning and tracking of visited destinations during the Erasmus period.

Travelling is a genuine part of the Erasmus experience, and this service reflects that. Students can create and manage trips, store relevant travel details, mark trips as completed, and keep a record of countries and cities they have visited. In the API-first version this is represented as a set of REST endpoints. In a future frontend, the visited countries data could be visualised on an interactive map. The service also interacts with the Notification Service via events such as TripCreated, TripUpdated, TripCompleted, CountryVisitedAdded, and TripReminderRequired.

6. Notification Service

Main responsibility: Send reminders and alerts based on events generated by the other microservices.

Rather than having every service implement its own notification logic — which would lead to code duplication and tight coupling — the Notification Service centralises all alert delivery. It listens to events published by the other services via the message broker and generates the appropriate notifications for the student. Events consumed by this service include ResidenceDeadlineApproaching, DocumentExpiryApproaching, RequiredDocumentMissing, UniversityAnnouncementCreated, and TripReminderRequired. Because business services only publish events and never call the Notification Service directly, they remain completely decoupled from the notification mechanism.

5. Communication Design

The system uses a hybrid communication strategy: synchronous REST for direct, user-initiated actions, and asynchronous event-driven messaging for background processes and cross-service notifications. This combination gives the platform the responsiveness of REST where it matters and the loose coupling of event-driven architecture where tight coupling would be a problem (Hohpe & Woolf, 2003).

Synchronous Communication (REST)

All client-facing interactions go through the API Gateway via REST APIs over HTTP with JSON payloads. When a student creates a profile, uploads a document, checks their residence permit status, adds a trip, or retrieves notifications, the request is routed to the responsible microservice. This layer is the primary demo surface and can be fully exercised through Postman. JWT-based authentication is enforced at the gateway level so that individual services do not need to handle authentication logic themselves.

Asynchronous Communication (Event-Driven Messaging)

For processes that do not require an immediate response — particularly anything involving the Notification Service — the system uses a message broker, either RabbitMQ or Apache Kafka. Services publish domain events when something relevant happens, and interested consumers react independently (Apache Software Foundation, 2024; RabbitMQ, 2024).

A concrete example of this flow: when the Smart Document Management Service detects that an insurance document expires in thirty days, it publishes a `DocumentExpiryApproaching` event to the broker. The Notification Service, which subscribes to this event type, receives it and sends an alert to the student. The Document Service never needs to know that a Notification Service exists, let alone how it works. This is the core benefit of this design pattern: it eliminates direct dependencies between services and makes the system easier to extend.

Internal Service Communication (gRPC)

For high-frequency, low-latency lookups between internal services, the system uses gRPC with Protocol Buffers instead of REST. The primary use case is profile data retrieval: several services — including the Residence Permit Service, the Smart Document Management Service, and the Notification Service — regularly need to look up student context such as exchange period dates, host university, or notification preferences. These are small, synchronous, typed queries that happen on nearly every operation. Using REST for these would introduce unnecessary overhead from HTTP header parsing and JSON serialisation. gRPC provides a faster and more type-safe alternative for this specific internal communication pattern (Google, 2023).

The Student Profile Service exposes a gRPC interface defined via a `.proto` file. Other services act as gRPC clients and call it directly when they need profile context. This keeps the profile lookup fast and schema-enforced, while REST continues to serve all client-facing and non-latency-sensitive paths. The result is a three-layer communication model: REST for external traffic, Kafka/RabbitMQ for async events, and gRPC for internal synchronous lookups.

Communication Summary — Service Interaction Table

The table below specifies, for each inter-service interaction in the system, which communication protocol is used and why.

From	To	Protocol	Why
Postman / Client	API Gateway → Any Service	REST	All user-initiated actions; primary demo surface
Residence Permit Service	Student Profile Service	gRPC	Fetch exchange dates for deadline urgency calculation
Residence Permit Service	Message Broker → Notification Service	Kafka Event	ResidenceDeadlineApproaching event
Smart Document Service	Student Profile Service	gRPC	Fetch profile context for document classification
Smart Document Service	Message Broker → Notification Service	Kafka Event	DocumentExpiryApproaching, RequiredDocumentMissing
University Services Service	Message Broker → Notification Service	Kafka Event	UniversityAnnouncementCreated, ExamDateChanged
Travel Planning Service	Message Broker → Notification Service	Kafka Event	TripReminderRequired event
Notification Service	Student Profile Service	gRPC	Fetch contact preferences before sending alert

6. Technology Stack

The following technologies were selected for their compatibility with the microservice architecture and with the course requirements:

- **Postman** API Testing and Demo:
- **Node.js (Student Profile, Travel Planning, Notification, University Services) — Python with FastAPI (Smart Document Management, Residence Permit)** Backend:
- **REST with JSON over HTTP** API Communication:
- **Spring Cloud Gateway or a custom gateway implementation** API Gateway:
- **RabbitMQ or Apache Kafka** Asynchronous Messaging:
- **gRPC (Protocol Buffers)** Internal Synchronous Service Communication (high-frequency, low-latency lookups):
- **PostgreSQL — one independent database per microservice** Databases:
- **JWT-based stateless authentication, enforced at the gateway** Authentication:
- **Docker and Docker Compose for local development and deployment** Containerisation:
- **OpenAPI / Swagger** API Documentation:
- **React.js — not part of the initial version** Optional Future Frontend:

Each microservice owns its own database schema with no shared tables across services. Inter-service communication happens exclusively through REST APIs or the message broker, never through direct database access. This ensures that each service can be developed, deployed, and scaled independently without creating hidden dependencies on other services' data structures (Richardson, 2018).

7. Architecture Diagram

The diagram below shows the full system architecture. Reading from top to bottom: Postman acts as the API client for the demo. Requests go through the API Gateway, which handles routing and authentication before forwarding to the appropriate microservice. The six microservices each own a separate database. Business services publish domain events to the message broker, and the Notification Service consumes those events to generate alerts. A future React.js frontend is indicated as an optional extension.

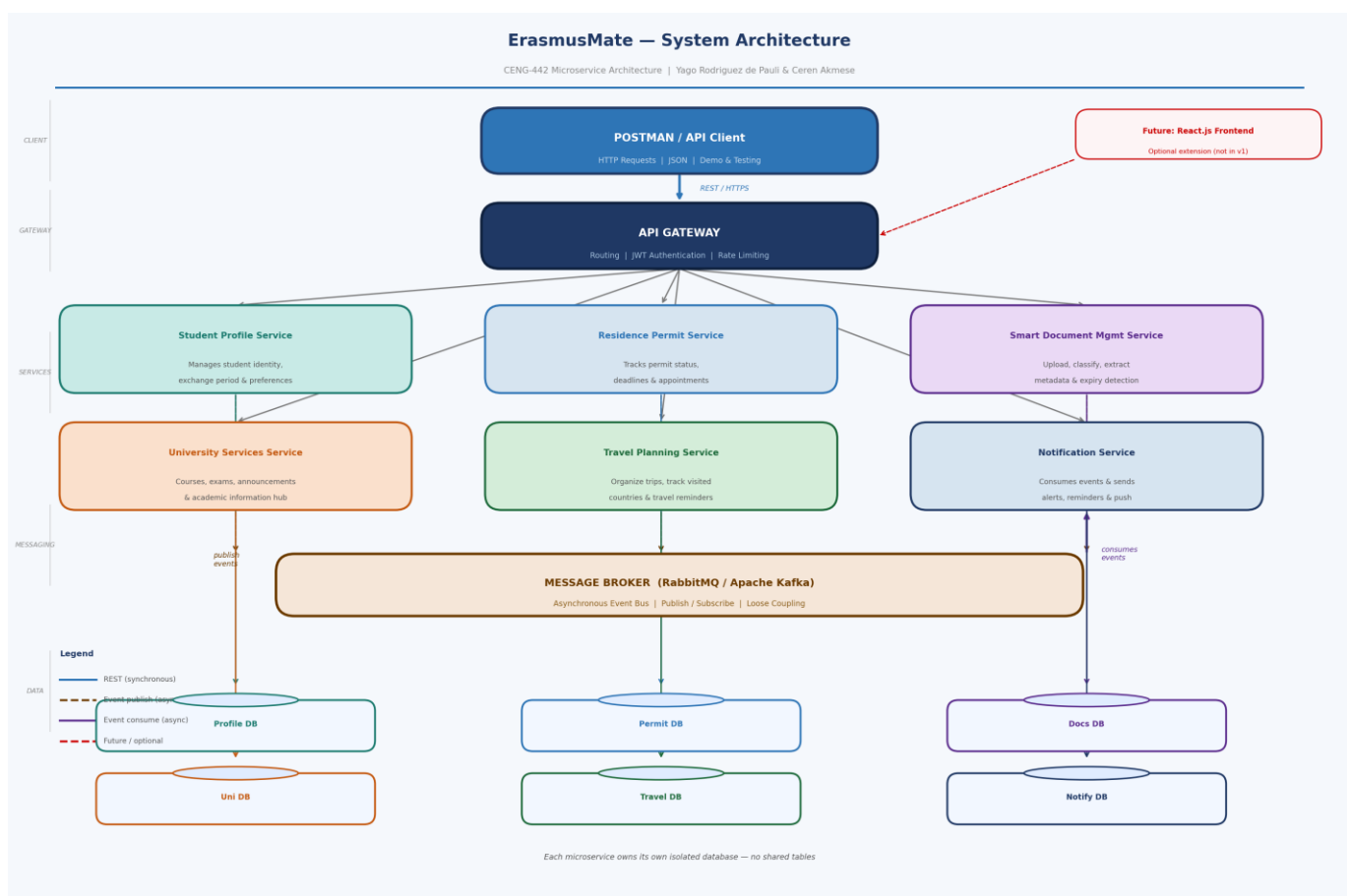


Diagram 1: Key architectural principles reflected in the diagram: each microservice is an independent node with its own database; the API Gateway is the single entry point for all client traffic; the message broker sits between publisher services and the Notification Service to ensure loose coupling; and there are no direct service-to-service database connections.

8. References

- Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.
- Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.
- Fowler, M., & Lewis, J. (2014). Microservices: A definition of this new architectural term. *martinfowler.com*. <https://martinfowler.com/articles/microservices.html>
- Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.
- Google. (2023). *gRPC Documentation*. <https://grpc.io/docs/>
- RabbitMQ. (2024). *RabbitMQ Documentation*. <https://www.rabbitmq.com/documentation.html>
- Apache Software Foundation. (2024). *Apache Kafka Documentation*. <https://kafka.apache.org/documentation/>